Scikit-Learn Cheat Sheet

A practical quick reference for beginners & intermediate users
Ordered from the most commonly used functions to the least

EVERY EXAMPLE IN THIS GUIDE USES THIS SETUP

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("HR_ListOfEmployees.csv")
df = df.dropna(subset=["Salary", "Performance_Rating"])

# the question: does this employee earn above the median?
df["High_Salary"] = (df["Salary"] > df["Salary"].median()).astype(int)

FEATURES = ["Years", "Is_Manager", "Dept_Band", "Age"]
X = df[FEATURES]          # the inputs
y = df["High_Salary"]     # the answer to learn

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

FOUR IDEAS THAT EXPLAIN MOST OF SCIKIT-LEARN

`X and y`	X holds the input columns; y is the single answer column.
`fit / predict`	Every model learns with .fit() then answers with .predict().
`train / test`	Score on data the model never saw, or you fool yourself.
`Pipeline`	Chains scaling and the model so test data never leaks in.

THE RUNNING EXAMPLE — AND HOW WELL IT WORKS

Task	`Predict if an employee earns above the median salary`
Rows used	`229 employees · 183 train / 46 test`
Baseline (always guess the biggest class)	`52.2% accuracy`
Decision tree, max_depth=3	`65.2% accuracy`
Logistic regression in a pipeline	`69.6% accuracy`

Blue bar = code you type **Green bar** = real output returned **119** examples · **15** topics

Every sample output was produced by actually running the code on scikit-learn 1.6. Companion to the Pandas & NumPy sheets.

Table of Contents

1. Setup & Loading Data

8 functions

Import scikit-learn and get data ready.

1.1

```
import sklearn    # Imports the library
```

SAMPLE OUTPUT

```
1.6.1
```

1.2

```
from sklearn.model_selection import train_test_split    # Imports only what you need
```

SAMPLE OUTPUT

```
train_test_split
```

1.3

```
df = pd.read_csv("HR_ListOfEmployees.csv")    # Loads your own CSV file
```

SAMPLE OUTPUT

```
(250, 14)
```

1.4

```
df.shape    # Rows and columns after cleaning
```

SAMPLE OUTPUT

```
(229, 18)
```

1.5

```
# Loads a practice dataset
from sklearn.datasets import load_iris
iris = load_iris()
```

SAMPLE OUTPUT

```
(150, 4)
```

1.6

```
load_iris(as_frame=True).frame.head(3)    # Practice data as a DataFrame
```

SAMPLE OUTPUT

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1               3.5                1.4               0.2       0
1                4.9               3.0                1.4               0.2       0
2                4.7               3.2                1.3               0.2       0
```

1.7

```
# Creates fake data for testing
from sklearn.datasets import make_classification
Xs, ys = make_classification(
    n_samples=100, n_features=4, random_state=0)
```

SAMPLE OUTPUT

```
((100, 4), (100))
```

1.8

```
df["High_Salary"].value_counts()    # Checks if classes are balanced
```

SAMPLE OUTPUT

```
High_Salary
0    117
1    112
```

2. Features & Target

9 functions

Choose your inputs (X) and answer (y).

2.1

```
# Picks the input columns
FEATURES = ["Years", "Is_Manager",
            "Dept_Band", "Age"]
X = df[FEATURES]
```

SAMPLE OUTPUT

```
   Years  Is_Manager  Dept_Band  Age
0      4           0          7   40
1      4           0          7   37
2      0           0          7   36
```

2.2

```
y = df["High_Salary"]    # Picks the column to predict
```

SAMPLE OUTPUT

```
0    0
1    1
2    1
3    1
```

2.3

```
X.shape, y.shape    # Inputs and target must match in rows
```

SAMPLE OUTPUT

```
((229, 4), (229))
```

2.4

```
# Builds a yes/no target column
df["High_Salary"] = (df["Salary"] >
                     df["Salary"].median()).astype(int)
```

SAMPLE OUTPUT

```
High_Salary
0    117
1    112
```

2.5

```
# Turns text into a 0/1 feature
df["Is_Manager"] = df["Position"].str.contains(
    "Manager").astype(int)
```

SAMPLE OUTPUT

```
Is_Manager
0    184
1     45
```

2.6

```
# Ranks departments by typical pay
band = df.groupby("Department")["Salary"].median().rank()
df["Dept_Band"] = df["Department"].map(band.astype(int))
```

SAMPLE OUTPUT

```
Department
Engineering        7
Finance            6
Human Resources    2
IT                 4
Marketing          5
Operations         1
Sales              3
```

2.7

```
y.mean().round(3)    # Share of the positive class
```

SAMPLE OUTPUT

```
0.489
```

2.8

```
X.isnull().sum()    # Models fail on missing values
```

SAMPLE OUTPUT

```
Years          0
Is_Manager     0
Dept_Band      0
Age            0
```

2.9

```
X.dtypes    # All features must be numeric
```

SAMPLE OUTPUT

```
Years          int32
Is_Manager     int64
Dept_Band      int64
Age            int64
```

3. Splitting the Data

8 functions

Hold back data to test honestly.

3.1

```
# Splits into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

SAMPLE OUTPUT

```
((183, 4), (46, 4))
```

3.2

```
X_train.shape, X_test.shape    # 80 percent train, 20 percent test
```

SAMPLE OUTPUT

```
((183, 4), (46, 4))
```

3.3

```
train_test_split(X, y, test_size=0.3, random_state=42)[1].shape    # Changes the test set size
```

SAMPLE OUTPUT

```
(69, 4)
```

3.4

```
# Keeps class balance in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

SAMPLE OUTPUT

```
(0.492, 0.478)
```

3.5

```
y_train.value_counts()    # Counts classes in the training set
```

SAMPLE OUTPUT

```
High_Salary
0    93
1    90
```

3.6

```
y_test.value_counts()    # Counts classes in the test set
```

SAMPLE OUTPUT

```
High_Salary
0    24
1    22
```

3.7

```
train_test_split(X, y, random_state=42, shuffle=False)[1].shape    # Splits without shuffling first
```

SAMPLE OUTPUT

```
(58, 4)
```

3.8

```
len(X_train) / len(X)    # Checks the split proportion
```

SAMPLE OUTPUT

```
0.799
```

4. Scaling & Encoding

9 functions

Put features on a fair scale.

4.1

```
# Rescales to mean 0, spread 1
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
```

SAMPLE OUTPUT

```
[[-1.15  1.99  0.76  0.45]
 [ 1.31 -0.5   1.24  1.52]
 [-0.84 -0.5  -0.19 -1.59]]
```

4.2

```
X_test_s = scaler.transform(X_test)    # Only transform test data, never fit
```

SAMPLE OUTPUT

```
[[-0.23 -0.5  -1.62 -1.39]
 [-0.84  1.99  0.76  0.16]
 [-1.46 -0.5   1.24 -1.78]]
```

4.3

```
scaler.mean_.round(2)    # Averages learned from training data
```

SAMPLE OUTPUT

```
[ 5.74  0.2   4.4  41.35]
```

4.4

```
# Squeezes values between 0 and 1
from sklearn.preprocessing import MinMaxScaler
MinMaxScaler().fit_transform(X_train)[:3].round(2)
```

SAMPLE OUTPUT

```
[[0.18 1.   0.83 0.65]
 [0.91 0.   1.   0.95]
 [0.27 0.   0.5  0.08]]
```

4.5

```
# Turns categories into 0/1 columns
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False)
enc.fit_transform(df[["Employment_Type"]])[:3]
```

SAMPLE OUTPUT

```
[[0. 0. 1.]
 [0. 1. 0.]
 [0. 1. 0.]]
```

4.6

```
enc.get_feature_names_out()    # Names of the new encoded columns
```

SAMPLE OUTPUT

```
['Employment_Type_Contractor' 'Employment_Type_Full-Time' 'Employment_Type_Part-Time']
```

4.7

```
# Turns labels into numbers
from sklearn.preprocessing import LabelEncoder
LabelEncoder().fit_transform(df["Department"])[:8]
```

SAMPLE OUTPUT

```
[0 0 0 0 0 0 0 0]
```

4.8

```
# Fills missing values automatically
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="mean")
imp.fit_transform(df[["Salary"]])[:3].round(0)
```

SAMPLE OUTPUT

```
[[ 69500.]
 [ 91500.]
 [100000.]]
```

4.9

```
# Different treatment per column type
from sklearn.compose import ColumnTransformer
pre = ColumnTransformer([
    ("num", StandardScaler(), FEATURES),
    ("cat", OneHotEncoder(), ["Department"])])
```

SAMPLE OUTPUT

```
(229, 11)
```

5. Training a Model

9 functions

Teach the model with .fit().

5.1

```
# Creates an untrained model
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
```

SAMPLE OUTPUT

```
(no value returned - the object is changed in place)
```

5.2

```
model.fit(X_train, y_train)    # Trains the model on your data
```

SAMPLE OUTPUT

```
LogisticRegression(max_iter=1000)
```

5.3

```
# Trains a decision tree
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
```

SAMPLE OUTPUT

```
DecisionTreeClassifier(max_depth=3, random_state=42)
```

5.4

```
# Trains many trees together
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

SAMPLE OUTPUT

```
RandomForestClassifier(random_state=42)
```

5.5

```
round(model.score(X_test, y_test), 4)    # Quick accuracy on test data
```

SAMPLE OUTPUT

```
0.6522
```

5.6

```
model.get_params()    # Shows the model settings
```

SAMPLE OUTPUT

```
{'C': 1.0, 'max_iter': 1000, 'penalty': l2, 'solver': lbfgs}
```

5.7

```
model.set_params(C=0.5)    # Changes a setting after creating
```

SAMPLE OUTPUT

```
0.5
```

5.8

```
model.classes_    # The classes the model learned
```

SAMPLE OUTPUT

```
[0 1]
```

5.9

```
model.n_features_in_    # How many features it expects
```

SAMPLE OUTPUT

```
4
```

6. Making Predictions

8 functions

Ask the trained model for answers.

6.1

```
preds = model.predict(X_test)    # Predicts labels for new data
```

SAMPLE OUTPUT

```
[0 1 0 1 0 1 1 0 1 1 0 1]
```

6.2

```
model.predict(X_test)[:8]    # First few predictions
```

SAMPLE OUTPUT

```
[0 1 0 1 0 1 1 0]
```

6.3

```
model.predict_proba(X_test)[:4].round(3)    # Confidence for each class
```

SAMPLE OUTPUT

```
[[0.638 0.362]
 [0.    1.   ]
 [0.667 0.333]
 [0.358 0.642]]
```

6.4

```
model.predict_proba(X_test)[:, 1][:6].round(3)    # Probability of the positive class
```

SAMPLE OUTPUT

```
[0.362 1.    0.333 0.642 0.362 1.   ]
```

6.5

```
# Predicts for one new employee
new = pd.DataFrame([[4, 1, 6, 35]], columns=FEATURES)
model.predict(new)
```

SAMPLE OUTPUT

```
[1]
```

6.6

```
model.predict(new)[0]    # Gets a single plain answer
```

SAMPLE OUTPUT

```
1
```

6.7

```
(preds == y_test).sum()    # Counts the correct predictions
```

SAMPLE OUTPUT

```
30
```

6.8

```
reg.predict(Xr_test)[:4].round(0)    # Predicts numbers, not classes
```

SAMPLE OUTPUT

```
[87780. 84900. 83533. 66119.]
```

7. Evaluating Classification

8 functions

Measure how good the answers are.

7.1

```
# Share of correct predictions
from sklearn.metrics import accuracy_score
round(accuracy_score(y_test, preds), 4)
```

SAMPLE OUTPUT

```
0.6522
```

7.2

```
# Right and wrong answers in a grid
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, preds)
```

SAMPLE OUTPUT

```
[[16  8]
 [ 8 14]]
```

7.3

```
# All the main scores at once
from sklearn.metrics import classification_report
print(classification_report(y_test, preds))
```

SAMPLE OUTPUT

```
              precision    recall  f1-score   support

           0       0.67      0.67      0.67        24
           1       0.64      0.64      0.64        22

    accuracy                           0.65        46
   macro avg       0.65      0.65      0.65        46
weighted avg       0.65      0.65      0.65        46
```

7.4

```
# Precision and recall scores
from sklearn.metrics import precision_score, recall_score
(round(precision_score(y_test, preds), 3),
 round(recall_score(y_test, preds), 3))
```

SAMPLE OUTPUT

```
(0.636, 0.636)
```

7.5

```
# Balance of precision and recall
from sklearn.metrics import f1_score
round(f1_score(y_test, preds), 3)
```

SAMPLE OUTPUT

```
0.636
```

7.6

```
# How well classes are separated
from sklearn.metrics import roc_auc_score
round(roc_auc_score(y_test,
      model.predict_proba(X_test)[:, 1]), 3)
```

SAMPLE OUTPUT

```
0.664
```

7.7

```
# Baseline your model must beat
from sklearn.dummy import DummyClassifier
dummy = DummyClassifier(strategy="most_frequent")
round(dummy.fit(X_train, y_train).score(X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.5217
```

7.8

```
# Confusion matrix with labels
pd.crosstab(y_test, preds,
            rownames=["Actual"], colnames=["Predicted"])
```

SAMPLE OUTPUT

```
Predicted   0   1
Actual
0          16   8
1           8  14
```

8. Evaluating Regression

8 functions

Score models that predict numbers.

8.1

```
# Share of variation explained
from sklearn.metrics import r2_score
round(r2_score(yr_test, rpreds), 4)
```

SAMPLE OUTPUT

```
0.4056
```

8.2

```
# Average error in real units
from sklearn.metrics import mean_absolute_error
round(mean_absolute_error(yr_test, rpreds))
```

SAMPLE OUTPUT

```
17239
```

8.3

```
# Punishes large errors more
from sklearn.metrics import mean_squared_error
round(mean_squared_error(yr_test, rpreds))
```

SAMPLE OUTPUT

```
439722719
```

8.4

```
np.sqrt(mean_squared_error(yr_test, rpreds)).round(0)    # Error in the original units
```

SAMPLE OUTPUT

```
20970.0
```

8.5

```
reg.coef_.round(1)    # Weight given to each feature
```

SAMPLE OUTPUT

```
[ 1600.  26885.7  5511.2  -296.9]
```

8.6

```
round(reg.intercept_, 1)    # Starting value of the line
```

SAMPLE OUTPUT

```
56006.0
```

8.7

```
pd.Series(reg.coef_.round(1), index=FEATURES)    # Coefficients with feature names
```

SAMPLE OUTPUT

```
Years          1600.0
Is_Manager    26885.7
Dept_Band      5511.2
Age            -296.9
```

8.8

```
round(reg.score(Xr_test, yr_test), 4)    # Same as R2 score
```

SAMPLE OUTPUT

```
0.4056
```

9. Cross-Validation

7 functions

Test on several splits, not just one.

9.1

```
# Scores from five different splits
from sklearn.model_selection import cross_val_score
cross_val_score(model, X, y, cv=5).round(3)
```

SAMPLE OUTPUT

```
[0.478 0.696 0.37  0.522 0.511]
```

9.2

```
cross_val_score(model, X, y, cv=5).mean().round(4)    # Average score across all folds
```

SAMPLE OUTPUT

```
0.5153
```

9.3

```
# Shuffles first if data is sorted
from sklearn.model_selection import StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cross_val_score(model, X, y, cv=cv).mean().round(4)
```

SAMPLE OUTPUT

```
0.6113
```

9.4

```
# Cross-validation for regression
from sklearn.model_selection import KFold
cross_val_score(reg, X, df["Salary"],
                cv=KFold(5, shuffle=True,
                         random_state=42)).mean().round(4)
```

SAMPLE OUTPUT

```
0.2955
```

9.5

```
# Cross-validate a chosen metric
cross_val_score(model, X, y, cv=5, scoring="f1").mean().round(4)
```

SAMPLE OUTPUT

```
0.4451
```

9.6

```
# Several metrics in one run
from sklearn.model_selection import cross_validate
res = cross_validate(model, X, y, cv=5,
                     scoring=["accuracy", "f1"])
res["test_f1"].round(3)
```

SAMPLE OUTPUT

```
[0.647 0.65  0.54  0.389 0.   ]
```

9.7

```
# Predictions from every fold
from sklearn.model_selection import cross_val_predict
cross_val_predict(model, X, y, cv=5)[:12]
```

SAMPLE OUTPUT

```
[1 1 1 1 1 1 1 1 1 1 1 1]
```

10. Pipelines

7 functions

Chain the steps into one safe object.

10.1

```
# Bundles preparation and model
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))])
```

SAMPLE OUTPUT

```
(no value returned - the object is changed in place)
```

10.2

```
pipe.fit(X_train, y_train)    # Trains every step in order
```

SAMPLE OUTPUT

```
0.6957
```

10.3

```
pipe.predict(X_test)[:10]    # Scales then predicts automatically
```

SAMPLE OUTPUT

```
[0 1 1 0 0 1 1 1 1 1]
```

10.4

```
# Shorter way to build a pipeline
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(),
                     LogisticRegression(max_iter=1000))
```

SAMPLE OUTPUT

```
[standardscaler, logisticregression]
```

10.5

```
pipe.named_steps["model"].coef_.round(3)    # Reaches inside a pipeline step
```

SAMPLE OUTPUT

```
[[ 0.277  0.661  0.671 -0.158]]
```

10.6

```
cross_val_score(pipe, X, y, cv=5).mean().round(4)    # Prevents leaking test information
```

SAMPLE OUTPUT

```
0.6373
```

10.7

```
# Full pipeline with mixed columns
from sklearn.compose import ColumnTransformer
full = Pipeline([("pre", pre), ("model", rf)])
round(full.fit(df, y).score(df, y), 4)
```

SAMPLE OUTPUT

```
0.9956
```

11. Tuning Hyperparameters

7 functions

Search for the best settings.

11.1

```
# Tries every combination of settings
from sklearn.model_selection import GridSearchCV
grid = {"max_depth": [2, 3, 5, 8]}
gs = GridSearchCV(model, grid, cv=5)
gs.fit(X, y)
```

SAMPLE OUTPUT

```
{'max_depth': 2}
```

11.2

```
gs.best_params_    # The winning settings
```

SAMPLE OUTPUT

```
{'max_depth': 2}
```

11.3

```
round(gs.best_score_, 4)    # Best average score found
```

SAMPLE OUTPUT

```
0.5414
```

11.4

```
gs.best_estimator_    # The ready-to-use best model
```

SAMPLE OUTPUT

```
DecisionTreeClassifier(max_depth=2, random_state=42)
```

11.5

```
# Compares every setting tried
pd.DataFrame(gs.cv_results_)[
    ["param_max_depth", "mean_test_score"]].round(3)
```

SAMPLE OUTPUT

```
   param_max_depth  mean_test_score
0                2            0.541
1                3            0.515
2                5            0.498
3                8            0.503
```

11.6

```
# Tries random settings, much faster
from sklearn.model_selection import RandomizedSearchCV
rs = RandomizedSearchCV(rf, {"n_estimators": [50, 100, 200],
                             "max_depth": [3, 5, None]},
                        n_iter=4, cv=3, random_state=42)
rs.fit(X, y)
```

SAMPLE OUTPUT

```
{'n_estimators': 50, 'max_depth': 3}
```

11.7

```
gs.predict(X_test)[:10]    # Best model predicts directly
```

SAMPLE OUTPUT

```
[0 1 1 1 0 1 1 0 1 1]
```

12. Common Models

9 functions

The models you will actually use.

12.1

```
# Yes/no prediction, fast and simple
from sklearn.linear_model import LogisticRegression
round(LogisticRegression(max_iter=1000).fit(
    X_train, y_train).score(X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.7174
```

12.2

```
# Predicts a number from features
from sklearn.linear_model import LinearRegression
round(LinearRegression().fit(Xr_train, yr_train).score(
    Xr_test, yr_test), 4)
```

SAMPLE OUTPUT

```
0.4056
```

12.3

```
# Easy to explain to non-experts
from sklearn.tree import DecisionTreeClassifier
round(DecisionTreeClassifier(max_depth=3, random_state=42).fit(
    X_train, y_train).score(X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.6522
```

12.4

```
# Strong all-round default choice
from sklearn.ensemble import RandomForestClassifier
round(RandomForestClassifier(n_estimators=100,
    random_state=42).fit(X_train, y_train).score(
    X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.6087
```

12.5

```
# Often the most accurate model
from sklearn.ensemble import GradientBoostingClassifier
round(GradientBoostingClassifier(random_state=42).fit(
    X_train, y_train).score(X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.5652
```

12.6

```
# Predicts using nearest neighbours
from sklearn.neighbors import KNeighborsClassifier
round(KNeighborsClassifier(n_neighbors=5).fit(
    X_train, y_train).score(X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.5217
```

12.7

```
# Good for complex boundaries
from sklearn.svm import SVC
round(SVC(random_state=42).fit(X_train, y_train).score(
    X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.5
```

12.8

```
# Very fast, good for text
from sklearn.naive_bayes import GaussianNB
round(GaussianNB().fit(X_train, y_train).score(
    X_test, y_test), 4)
```

SAMPLE OUTPUT

```
0.6957
```

12.9

```
# Forest for predicting numbers
from sklearn.ensemble import RandomForestRegressor
round(RandomForestRegressor(n_estimators=100,
    random_state=42).fit(Xr_train, yr_train).score(
    Xr_test, yr_test), 4)
```

SAMPLE OUTPUT

```
0.1768
```

13. Feature Importance

8 functions

See which columns actually matter.

13.1

```
model.feature_importances_.round(3)    # How much each feature helped
```

SAMPLE OUTPUT

```
[0.288 0.082 0.216 0.415]
```

13.2

```
# Importance with feature names
pd.Series(model.feature_importances_.round(3),
          index=FEATURES).sort_values(ascending=False)
```

SAMPLE OUTPUT

```
Age           0.415
Years         0.288
Dept_Band     0.216
Is_Manager    0.082
```

13.3

```
model.coef_.round(3)    # Feature weights in linear models
```

SAMPLE OUTPUT

```
[[ 0.277  0.661  0.671 -0.158]]
```

13.4

```
# Importance that works on any model
from sklearn.inspection import permutation_importance
r = permutation_importance(model, X_test, y_test,
                           n_repeats=5, random_state=42)
pd.Series(r.importances_mean.round(3), index=FEATURES)
```

SAMPLE OUTPUT

```
Years         0.070
Is_Manager    0.022
Dept_Band     0.022
Age           0.061
```

13.5

```
# Keeps only the best features
from sklearn.feature_selection import SelectKBest, f_classif
sel = SelectKBest(f_classif, k=2).fit(X, y)
sel.get_feature_names_out()
```

SAMPLE OUTPUT

```
['Is_Manager' 'Dept_Band']
```

13.6

```
sel.scores_.round(2)    # Score given to each feature
```

SAMPLE OUTPUT

```
[ 1.83 16.95 18.85  0.28]
```

13.7

```
# Removes weak features step by step
from sklearn.feature_selection import RFE
RFE(model, n_features_to_select=2).fit(
    X, y).get_feature_names_out()
```

SAMPLE OUTPUT

```
['Is_Manager' 'Dept_Band']
```

13.8

```
X.corrwith(y).round(3)    # Simple correlation with the target
```

SAMPLE OUTPUT

```
Years         0.089
Is_Manager    0.264
Dept_Band     0.277
Age          -0.035
```

14. Clustering & PCA

8 functions

Find patterns without any labels.

14.1

```
# Groups rows into clusters
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42, n_init=10)
km.fit(X_s)
```

SAMPLE OUTPUT

```
[0 0 0 0 0 1 0 0 0 0 1 0]
```

14.2

```
km.labels_[:12]    # Which cluster each row joined
```

SAMPLE OUTPUT

```
[0 0 0 0 0 1 0 0 0 0 1 0]
```

14.3

```
pd.Series(km.labels_).value_counts()    # How many rows per cluster
```

SAMPLE OUTPUT

```
0    92
1    92
2    45
```

14.4

```
km.cluster_centers_.round(2)    # The centre of each cluster
```

SAMPLE OUTPUT

```
[[-0.8  -0.49  0.23 -0.37]
 [ 0.78 -0.49 -0.17  0.37]
 [ 0.04  2.02 -0.13  0.  ]]
```

14.5

```
# Checks how clean the clusters are
from sklearn.metrics import silhouette_score
round(silhouette_score(X_s, km.labels_), 3)
```

SAMPLE OUTPUT

```
0.26
```

14.6

```
# Squeezes features into two columns
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
pca.fit_transform(X_s)[:3].round(2)
```

SAMPLE OUTPUT

```
[[-1.32  0.34]
 [-1.36  0.1 ]
 [-2.19 -0.35]]
```

14.7

```
pca.explained_variance_ratio_.round(3)    # Information kept by each part
```

SAMPLE OUTPUT

```
[0.293 0.264]
```

14.8

```
round(pca.explained_variance_ratio_.sum(), 3)    # Total information kept
```

SAMPLE OUTPUT

```
0.558
```

15. Saving & Loading Models

6 functions

Keep a trained model for later.

15.1

```
# Saves the trained model to disk
import joblib
joblib.dump(model, "hr_model.joblib")
```

SAMPLE OUTPUT

```
[hr_model.joblib]
```

15.2

```
loaded = joblib.load("hr_model.joblib")    # Loads the model back again
```

SAMPLE OUTPUT

```
0.6522
```

15.3

```
loaded.predict(X_test)[:10]    # Loaded model predicts as before
```

SAMPLE OUTPUT

```
[0 1 0 1 0 1 1 0 1 1]
```

15.4

```
joblib.dump(pipe, "hr_pipeline.joblib")    # Save the whole pipeline, not just the model
```

SAMPLE OUTPUT

```
[hr_pipeline.joblib]
```

15.5

```
# Checks the saved file size in bytes
import os
os.path.getsize("hr_model.joblib")
```

SAMPLE OUTPUT

```
2633
```

15.6

```
# Shows models as plain text
from sklearn import set_config
set_config(display="text")
```

SAMPLE OUTPUT

```
models now print as text, not diagrams
```